

CENTRO DE INGENIERÍA DE SOFTWARE (CIS)
PROTECCIÓN S.A.

Prueba Técnica AI-First

REPORTE DE AUDITORÍA DE SEGURIDAD DE CÓDIGO

Reto A · Revisión de Merge Request

Módulo de registro de aportes a un fondo voluntario

Candidato	Santiago Manco Maya
Entregable	Reto A — Auditoría de código (hallazgos, severidad y priorización de MR)
Alcance	Módulo reto-a (Spring Boot · Java 21 · JPA · H2)
Contexto	Entorno financiero regulado por la Superintendencia Financiera de Colombia (SFC)
Fecha	8 de octubre de 2026
Versión	1.0
Clasificación	Confidencial

Contenido

1. Resumen ejecutivo	3
Resultados en cifras	3
Lo más importante	3
Las tres acciones prioritarias	3
2. Matriz resumen de hallazgos	4
3. Hallazgos en detalle	5
A-01 · Inyección SQL en la consulta de consolidado	6
A-02 · Ausencia de autenticación y autorización (IDOR)	7
A-03 · Evasión del tope mensual por comparación con «==»	8
A-04 · Uso de «double» para representar valores monetarios	9
A-05 · Falta de atomicidad y control de concurrencia en el saldo	10
A-06 · La operación de registro no es idempotente	11
A-07 · El saldo no está segmentado por periodo mensual	12
A-08 · Validación de entrada insuficiente	13
A-09 · Manejo de errores deficiente y fuga de información	14
A-10 · Registro de datos financieros sensibles en logs	15
A-11 · Exposición de la entidad JPA en la API	16
A-12 · Violación de límites arquitectónicos (Clean Architecture / CQRS)	17
A-13 · Trazabilidad y auditoría débiles del evento	18
4. Conclusiones	19
Fortalezas	19
Debilidades	19
Veredicto del Merge Request	19
Qué falta para ser apto para producción en un entorno SFC	19

1. Resumen ejecutivo

Se auditó el módulo reto-a, un servicio de registro de aportes a un fondo voluntario, revisándolo como si fuera un Merge Request de una célula del CIS. El módulo compila y los tests «felices» pasan; aun así contiene defectos que un revisor bloquearía antes de aprobar, varios de ellos sobre la corrección numérica, la concurrencia, la idempotencia y la seguridad del manejo de dinero en un entorno regulado por la SFC.

Resultados en cifras

Hallazgos por severidad	Críticos 2 · Altos 6 · Medios 5 (de 8 identificados; se detallan los 5 primeros)
Nivel de riesgo global	Alto
Juicio de Merge Request	No aprobar: A-01 a A-08 son bloqueantes
Metodología de severidad	CVSS v3.1 (calculadora de FIRST); OWASP Top 10 y CWE para clasificar

Nota de criterio: la severidad sigue el puntaje CVSS v3.1. Por eso algunos defectos que, por su impacto de negocio en un entorno SFC, un revisor trataría como críticos (p. ej. la lógica rota del tope o el uso de double para dinero) aparecen clasificados como Altos. El veredicto de MR se guía por la criticidad de bloqueo, no solo por el puntaje.

Lo más importante

- El módulo manipula dinero y corre en un contexto regulado, pero expone una inyección SQL en la consulta de consolidado y no tiene ninguna autenticación: ambos son explotables sin privilegios.
- La regla central del negocio —el tope mensual— no funciona: se valida con «==» en lugar de «>», el dinero se maneja con double y el saldo no se segmenta por mes, de modo que el tope prácticamente nunca bloquea.
- Falta lo que el enunciado pide de forma explícita: idempotencia en el registro y consistencia bajo concurrencia (no hay transacción ni bloqueo), lo que produce descuadres y doble conteo ante reintentos normales.

Las tres acciones prioritarias

1. Parametrizar la consulta de consolidado y exigir autenticación y autorización en ambos endpoints (A-01, A-02).
2. Corregir la lógica del tope (de «==» a «>»), migrar el dinero a BigDecimal y envolver el registro en una transacción con control de concurrencia (A-03, A-04, A-05).
3. Hacer idempotente el registro y segmentar el saldo por periodo mensual (A-06, A-07).

Nota metodológica (uso de IA). La auditoría se apoyó en IA para acelerar la lectura del código, pero cada hallazgo se verificó manualmente. Los defectos numéricos se confirmaron reproduciéndolos en Java 21: el error de precisión de double ($0,1 \times 10 = 0,9999999999999999$), el caso Infinity desde 1e400 y el comportamiento del tope con «==» sobre los saldos de data.sql (AF-003).

2. Matriz resumen de hallazgos

Una fila por hallazgo; coincide con las fichas de la sección 3. Ordenada por severidad (críticos, altos y medios). Para los medios se detallan los cinco primeros de los ocho identificados.

ID	Hallazgo	Componente	OWASP / CWE	CVSS	Severidad
A-01	Inyección SQL en la consulta de consolidado	AporteController	A03 / CWE-89	9.8	Crítica
A-02	Ausencia de autenticación y autorización (IDOR)	AporteController	A01 / CWE-862	9.1	Crítica
A-03	Evasión del tope mensual por comparación con «==»	AporteService	A04 / CWE-697	7.5	Alta
A-04	Uso de «double» para representar valores monetarios	Modelo (double)	A04 / CWE-1339	7.5	Alta
A-05	Falta de atomicidad y control de concurrencia en el saldo	AporteService	A04 / CWE-362	7.5	Alta
A-06	La operación de registro no es idempotente	POST /aportes	A04 / CWE-837	7.5	Alta
A-07	El saldo no está segmentado por periodo mensual	Saldo / Repo	A04 / CWE-840	7.5	Alta
A-08	Validación de entrada insuficiente	AporteRequest	A04 / CWE-20	7.5	Alta
A-09	Manejo de errores deficiente y fuga de información	Controller/Advice	A05 / CWE-209	5.3	Media
A-10	Registro de datos financieros sensibles en logs	AporteService	A09 / CWE-532	4.3	Media
A-11	Exposición de la entidad JPA en la API	AporteController	A08 / CWE-915	5.3	Media
A-12	Violación de límites arquitectónicos (Clean Architecture / CQRS)	Arquitectura	A04 / CWE-710	4.0	Media
A-13	Trazabilidad y auditoría débiles del evento	EventoAporte	A09 / CWE-778	4.3	Media

Escala de severidad (CVSS v3.1): Crítica 9,0–10,0 · Alta 7,0–8,9 · Media 4,0–6,9 · Baja 0,1–3,9.

3. Hallazgos en detalle

Cada hallazgo usa la misma ficha, con su descripción, clasificación, acciones correctivas, fragmento de código relevante y un espacio reservado para la evidencia. Orden: críticos (A-01–A-02), altos (A-03–A-08) y medios (A-09–A-13).

Cómo leer una ficha

Riesgo / CVSS	Severidad, puntaje base y vector.
Hallazgo	Qué se encontró, de forma concreta.
Activos afectados	Clase, método y línea donde está.
Escenario de riesgo	Cómo se abusa o se manifiesta.
Impacto / Probabilidad	Qué se pierde y qué tan fácil ocurre.
Acciones correctivas	Arreglo inmediato y solución estructural.

A-01 · Inyección SQL en la consulta de consolidado

Riesgo / CVSS v3.1	Severidad: Crítica · Base Score: 9.8 CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Hallazgo	El endpoint de consulta construye la sentencia SQL concatenando directamente los parámetros afiliadoId y periodo recibidos del cliente, sin parametrizar.
Activos afectados	AporteController.java · método consolidado() · líneas 38–44.
Escenario de riesgo	Un atacante envía afiliadoId=' OR '1'='1 y obtiene los aportes de todos los afiliados. Con UNION o consultas apiladas puede leer otras tablas o, según el driver y los privilegios, alterar datos.
Impacto	Fuga masiva de datos financieros de todos los afiliados y posible manipulación de la base de datos. En un entorno regulado por la SFC implica violación de confidencialidad y de habeas data (Ley 1581).
Probabilidad	Alta — la carga es trivial, no requiere privilegios ni credenciales.
Clasificación	OWASP: A03:2021 – Inyección · CWE: CWE-89 (Inyección SQL) · ATT&CK: T1190 – Explotación de aplicación expuesta
Validación	Verdadero positivo. Defecto evidente por lectura de código; patrón de concatenación reproducible con cualquier cliente HTTP.
Descripción y causa raíz	La concatenación de entrada no confiable dentro del SQL permite alterar la estructura de la consulta. Agrava el defecto que el controlador acceda directamente a la base de datos con JdbcTemplate, saltándose el servicio y el repositorio.
Superficie de ataque	Parámetros de query afiliadoId y periodo del endpoint GET /api/aportes/consolidado.
Acciones correctivas	Inmediata: Usar una consulta parametrizada (binding de parámetros). Mejor aún: reutilizar findByAfiliadoIdAndPeriodo del AporteJpaRepository, que ya existe en el proyecto y no se usa. Estructural: Prohibir SQL dinámico por concatenación; análisis estático (SAST) que detecte concatenación de SQL; revisión obligatoria en el MR.

Evidencia

```
38  @GetMapping("/consolidado")
39  public List<Aporte> consolidado(@RequestParam String afiliadoId,
40                                @RequestParam String periodo) {
41      String sql = "SELECT * FROM aporte WHERE afiliado_id = '"
42                  + afiliadoId + "' AND periodo = '" + periodo + "'";
43      return jdbc.query(sql, aporteRowMapper);
44  }
45  }
46
```

A-02 · Ausencia de autenticación y autorización (IDOR)

Riesgo / CVSS v3.1	Severidad: Crítica · Base Score: 9.1 CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N
Hallazgo	Ningún endpoint exige autenticación ni verifica que el afiliado de la sesión/token coincida con el afiliadold del recurso solicitado.
Activos afectados	AporteController.java · POST /api/aportes y GET /api/aportes/consolidado (ambos endpoints).
Escenario de riesgo	Cualquiera registra aportes a nombre de cualquier afiliado o consulta el consolidado de cualquier afiliado con solo cambiar el identificador (referencia directa a objeto insegura).
Impacto	Acceso y escritura no autorizados sobre los datos financieros de cualquier afiliado; suplantación de identidad en el registro de aportes.
Probabilidad	Alta — basta con llamar al endpoint; no hay barrera alguna.
Clasificación	OWASP: A01:2021 – Pérdida de control de acceso · CWE: CWE-862 (Falta de autorización); relacionado con CWE-639 (IDOR) · ATT&CK: T1190 – Explotación de aplicación expuesta
Validación	Verdadero positivo. Ausencia total de configuración de seguridad verificable por lectura de código y del pom.xml.
Descripción y causa raíz	La API no integra Spring Security ni ningún control de acceso. El afiliadold viaja como dato del cliente y se confía en él sin comprobar la propiedad del recurso.
Superficie de ataque	Todos los endpoints del controlador y el parámetro afiliadold.
Acciones correctivas	Inmediata: Exigir autenticación (OAuth2/JWT con Spring Security) y verificar que el afiliado autenticado sea el dueño del recurso antes de leer o escribir. Estructural: Control de acceso por objeto en cada petición; si la autenticación la provee un API gateway, documentarlo explícitamente como supuesto de arquitectura.

Evidencia

```
@PostMapping
public Aporte registrar(@RequestBody AporteRequest req) {
    return service.registrar(req);
}

@GetMapping("/consolidado")
public List<Aporte> consolidado(@RequestParam String afiliadoId,
                                @RequestParam String periodo) {
    String sql = "SELECT * FROM aporte WHERE afiliado_id = '"
        + afiliadoId + "' AND periodo = '" + periodo + "'";
    return jdbc.query(sql, aporteRowMapper);
}
```

A-03 · Evasión del tope mensual por comparación con «==»

Riesgo / CVSS v3.1	Severidad: Alta · Base Score: 7.5 CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:N
Hallazgo	La validación del tope usa «==» en lugar de «>». Solo rechaza cuando el acumulado queda exactamente en el tope; deja pasar cualquier valor que lo supere y, de paso, rechaza aportes legítimos que caen justo en el límite.
Activos afectados	AporteService.java · método registrar() · línea 45.
Escenario de riesgo	Reproducido con AF-003 (saldo inicial 4.500.000, tope 10.000.000): un aporte de 5.500.000 (que llega exactamente a 10.000.000) se rechaza, mientras que un aporte de 6.000.000 (que llega a 10.500.000) se acepta y supera el tope.
Impacto	Se incumple una regla de negocio regulada: el tope mensual deja de ser efectivo. Afecta la integridad del control financiero.
Probabilidad	Alta — se dispara con valores normales de operación, sin intención maliciosa.
Clasificación	OWASP: A04:2021 – Diseño inseguro · CWE: CWE-697 (Comparación incorrecta); relacionado con CWE-840 (Error de lógica de negocio) · ATT&CK: No aplica
Validación	Verdadero positivo. Comportamiento reproducido en Java 21 con los valores de data.sql.
Descripción y causa raíz	Comparar igualdad exacta sobre un acumulado no cubre el caso «mayor que». El uso de double (ver A-04) hace además que la igualdad casi nunca se cumpla, de modo que el tope prácticamente nunca bloquea.
Superficie de ataque	Lógica de validación del monto en el caso de uso de registro.
Acciones correctivas	Inmediata: Cambiar a una comparación «mayor que» sobre BigDecimal (p. ej. nuevo.compareTo(tope) > 0) y definir explícitamente si el tope es inclusivo o exclusivo. Estructural: Pruebas unitarias que cubran el límite (debajo, en el tope y por encima); la ausencia de este caso en la suite actual ocultó el defecto.

Fragmento relevante

```
double nuevo = s.getTotalMes() + monto;
if (nuevo == topeMensual) {                // debería ser nuevo > tope
    throw new IllegalArgumentException("El monto supera el tope mensual permitido");
}
```

A-04 · Uso de «double» para representar valores monetarios

Riesgo / CVSS v3.1	Severidad: Alta · Base Score: 7.5 CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:N
Hallazgo	El dinero se maneja con double. Hay errores de representación (sumar 0,1 diez veces da 0,9999999999999999) y las comparaciones de igualdad no son confiables. Además, un monto como 1e400 en el JSON se deserializa como Infinity.
Activos afectados	Aporte.monto, Saldo.totalMes, EventoAporte.monto, AporteRequest.monto y los parámetros @Value (topeMensual, umbralRevision), todos de tipo double.
Escenario de riesgo	Infinity supera la validación monto<=0 y, al sumarse al saldo, lo corrompe y anula el tope de ahí en adelante; los errores de redondeo producen descuadres acumulados.
Impacto	Descuadres financieros sistemáticos e imposibilidad de cuadrar saldos; inaceptable en un contexto SFC.
Probabilidad	Media–Alta — el error de precisión es permanente; el caso Infinity es inducible por el cliente.
Clasificación	OWASP: A04:2021 – Diseño inseguro · CWE: CWE-1339 (Precisión/exactitud insuficiente de números reales); relacionado con CWE-682 (Cálculo incorrecto) · ATT&CK: No aplica
Validación	Verdadero positivo. Comportamiento de double, NaN e Infinity reproducido en Java 21.
Descripción y causa raíz	El tipo double es de punto flotante binario y no representa exactamente los decimales en base 10. No debe usarse para dinero.
Superficie de ataque	Todos los campos y cálculos monetarios del módulo.
Acciones correctivas	Inmediata: Migrar a BigDecimal con escala fija y modo de redondeo explícito; columna NUMERIC(19,2) en la base de datos. Estructural: Un value object Monto que encapsule moneda, escala y reglas de redondeo, y rechace valores no finitos.

Fragmento relevante

```
// 0.1 sumado 10 veces -> 0.9999999999999999
// (double) 1e400      -> Infinity (pasa el chequeo monto <= 0)
private double monto;  // debería ser java.math.BigDecimal
```

A-05 · Falta de atomicidad y control de concurrencia en el saldo

Riesgo / CVSS v3.1	Severidad: Alta · Base Score: 7.5 CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:N
Hallazgo	Un mismo bloque arrastra dos problemas: (1) atomicidad — se persisten saldo, evento y aporte en operaciones separadas sin transacción; si falla la última, el saldo ya quedó incrementado sin que exista el aporte; (2) concurrencia — el patrón leer-sumar-guardar no tiene bloqueo.
Activos afectados	AporteService.java · método registrar() · líneas 40–66 (sin @Transactional).
Escenario de riesgo	Un fallo entre la escritura del saldo y la del aporte deja la cuenta descuadrada. Dos aportes concurrentes leen el mismo saldo, ambos suman sobre el valor viejo y uno de los incrementos se pierde (lost update), pudiendo además superar el tope.
Impacto	Pérdida de integridad del saldo: dinero descuadrado y topes evadibles bajo concurrencia.
Probabilidad	Alta — la concurrencia es trivial de inducir y no hay ningún candado.
Clasificación	OWASP: A04:2021 – Diseño inseguro · CWE: CWE-362 (Condición de carrera); relacionado con CWE-662 (Sincronización incorrecta) · ATT&CK: No aplica
Validación	Verdadero positivo. Defecto estructural evidente por lectura de código.
Descripción y causa raíz	Sin @Transactional cada save() se confirma por separado. Sin bloqueo pesimista ni versión optimista, no hay garantía de aislamiento entre peticiones simultáneas.
Superficie de ataque	Lectura y actualización del saldo en el registro de aportes.
Acciones correctivas	Inmediata: Anotar el caso de uso con @Transactional (todo o nada) y proteger el saldo con @Lock(PESSIMISTIC_WRITE) o @Version (bloqueo optimista). Estructural: Preferir una actualización atómica condicionada al tope (UPDATE ... WHERE total + :monto <= :tope) para resolver consistencia y límite en una sola operación.

Fragmento relevante

```
s.setTotalMes(nuevo);
saldoRepo.save(s);           // commit #1
...
eventoRepo.save(new EventoAporte(aporte)); // commit #2
return aporteRepo.save(aporte); // commit #3 (si falla, el saldo ya subió)
```

A-06 · La operación de registro no es idempotente

Riesgo / CVSS v3.1	Severidad: Alta · Base Score: 7.5 CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:N
Hallazgo	El enunciado exige que el registro sea idempotente y no lo es. No hay clave de idempotencia ni control de duplicados.
Activos afectados	POST /api/aportes (AporteController + AporteService).
Escenario de riesgo	Un reintento del canal (por ejemplo, un timeout en la app móvil que repite la petición) registra el aporte dos veces y suma dos veces al saldo y al tope.
Impacto	Doble conteo de aportes y descuadre del saldo ante reintentos normales de red.
Probabilidad	Alta — los reintentos son comunes en integraciones de canales.
Clasificación	OWASP: A04:2021 – Diseño inseguro · CWE: CWE-837 (Falta de refuerzo de una acción única) · ATT&CK: No aplica
Validación	Verdadero positivo. Requisito explícito no implementado.
Descripción y causa raíz	Sin un identificador de operación que el servidor reconozca, dos peticiones idénticas producen dos efectos.
Superficie de ataque	El endpoint de registro y la ausencia de una restricción de unicidad.
Acciones correctivas	Inmediata: Aceptar un encabezado Idempotency-Key, con restricción de unicidad en base de datos; ante un repetido, devolver la respuesta original sin volver a registrar. Estructural: Diseñar la idempotencia a escala (ventana de deduplicación, almacenamiento de respuestas) como parte del contrato del canal.

A-07 · El saldo no está segmentado por periodo mensual

Riesgo / CVSS v3.1	Severidad: Alta · Base Score: 7.5 CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:N
Hallazgo	El saldo se busca solo por afiliadold e ignora el mes, de modo que el acumulado nunca se reinicia: el «tope mensual» es en la práctica un tope de por vida. Además, data.sql carga saldos de 2025-06 mientras el aporte toma el periodo de LocalDate.now().
Activos afectados	SaldoJpaRepository.findByAfiliadold, Saldo.mes y AporteService (líneas 40–50).
Escenario de riesgo	El acumulado de meses anteriores se arrastra y bloquea aportes del mes en curso. Si existiera más de una fila de saldo por afiliado, el findByAfiliadold que devuelve Optional lanzaría IncorrectResultSizeDataAccessException.
Impacto	La regla central del módulo (tope mensual) no funciona; comportamiento incorrecto y frágil.
Probabilidad	Alta — se manifiesta en operación normal al cambiar de mes.
Clasificación	OWASP: A04:2021 – Diseño inseguro · CWE: CWE-840 (Error de lógica de negocio) · ATT&CK: No aplica
Validación	Verdadero positivo. Discrepancia entre data.sql (2025-06) y el periodo calculado con now().
Descripción y causa raíz	El modelo de saldo tiene un campo mes que no se usa en la búsqueda ni en la escritura. La tabla saldo cumple además, de forma implícita, la validación de existencia del afiliado, mezclando dos responsabilidades.
Superficie de ataque	Consulta y actualización del saldo mensual.
Acciones correctivas	Inmediata: Buscar y actualizar por la pareja (afiliadold, mes); crear el registro del mes si no existe; añadir una restricción de unicidad en (afiliado_id, mes). Estructural: Separar la existencia del afiliado de su saldo; definir claramente el periodo de corte y su zona horaria.

Fragmento relevante

```
Optional<Saldo> findByAfiliadoId(String afiliadoId); // ignora el mes
// correcto: findByAfiliadoIdAndMes(String afiliadoId, String mes)
```

A-08 · Validación de entrada insuficiente

Riesgo / CVSS v3.1	Severidad: Alta · Base Score: 7.5 CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:N
Hallazgo	No hay @Valid ni anotaciones de validación. afiliadoId nulo llega hasta el repositorio; canal admite texto libre; y un monto Infinity (desde 1e400) supera el único chequeo existente (monto<=0).
Activos afectados	AporteRequest.java (sin Bean Validation) y AporteService.registrar().
Escenario de riesgo	Entradas malformadas provocan errores no controlados o, en el caso de Infinity, corrompen el saldo y anulan el tope.
Impacto	Errores no gestionados e integridad del saldo comprometida por valores no finitos.
Probabilidad	Media — requiere construir la petición, pero es trivial.
Clasificación	OWASP: A04:2021 – Diseño inseguro · CWE: CWE-20 (Validación de entrada incorrecta) · ATT&CK: No aplica
Validación	Verdadero positivo. Comportamiento de Infinity reproducido en Java 21.
Descripción y causa raíz	La validación se limita a comprobar monto<=0 en el servicio, sin cubrir nulos, formato ni rangos, y sin un canal controlado.
Superficie de ataque	Cuerpo del POST (afiliadoId, monto, canal).
Acciones correctivas	Inmediata: Aplicar Bean Validation (@NotBlank en afiliadoId, @Positive y @Digits en monto, un enum Canal) y @Valid en el controlador. Estructural: Revalidar las invariantes también en la capa de dominio, no solo en el borde.

Fragmento relevante

```
@Data @NoArgsConstructor @AllArgsConstructor
public class AporteRequest {
    private String afiliadoId;    // sin @NotBlank
    private double monto;        // sin @Positive / @Digits
    private String canal;        // texto libre, debería ser un enum
}
```

A-09 · Manejo de errores deficiente y fuga de información

Riesgo / CVSS v3.1	Severidad: Media · Base Score: 5.3 CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Hallazgo	Los <code>IllegalArgumentException</code> se propagan como HTTP 500 en vez de 400/404/422; no hay manejador global; y el mensaje «Afiliado no encontrado: » + <code>afiliadoId</code> devuelve el input del usuario tal cual.
Activos afectados	<code>AporteController</code> (sin <code>@RestControllerAdvice</code>) y <code>AporteService</code> (lanza <code>IllegalArgumentException</code>).
Escenario de riesgo	El cliente recibe trazas o códigos incorrectos que dificultan el consumo y pueden revelar detalles internos.
Impacto	Semántica HTTP incorrecta y posible fuga de información; mala experiencia de integración.
Probabilidad	Media — ocurre con cualquier entrada inválida.
Clasificación	OWASP: A05:2021 – Configuración de seguridad incorrecta · CWE: CWE-209 (Exposición de información por mensajes de error); relacionado con CWE-755 · ATT&CK: No aplica
Validación	Verdadero positivo. Evidente por lectura de código.
Descripción y causa raíz	La ausencia de un manejador de excepciones centralizado deja que las excepciones de negocio escapen como errores de servidor.
Superficie de ataque	Respuestas de error de ambos endpoints.
Acciones correctivas	Inmediata: Añadir un <code>@RestControllerAdvice</code> que mapee las excepciones de negocio a códigos adecuados y a mensajes genéricos sin eco del input. Estructural: Definir un contrato de errores (tipo <code>ProblemDetail</code> / RFC 7807) consistente en todo el servicio.

A-10 · Registro de datos financieros sensibles en logs

Riesgo / CVSS v3.1	Severidad: Media · Base Score: 4.3 CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:L/I:N/A:N
Hallazgo	Se registra en texto plano el monto y el identificador del afiliado en cada aporte.
Activos afectados	AporteService.java · línea 64.
Escenario de riesgo	Quien tenga acceso a los logs accede a datos financieros y personales sin control adicional.
Impacto	Exposición de datos personales y financieros en logs; tensión con el régimen de habeas data (Ley 1581).
Probabilidad	Baja–Media — requiere acceso a los logs.
Clasificación	OWASP: A09:2021 – Fallos de registro y monitoreo · CWE: CWE-532 (Inserción de información sensible en archivos de log) · ATT&CK: No aplica
Validación	Verdadero positivo. Evidente por lectura de código.
Descripción y causa raíz	Los logs se tratan como contenido no sensible; los datos de negocio no se enmascaran.
Superficie de ataque	Mensajes de log del servicio.
Acciones correctivas	Inmediata: Enmascarar o eliminar monto y afiliado de los logs; registrar en su lugar un identificador de correlación. Estructural: Política de clasificación de datos y de logging; revisión de qué se registra en producción.

Fragmento relevante

```
log.info("Aporte registrado: monto={ } afiliado={ }", monto, req.getAfiliadoId());
```

A-11 · Exposición de la entidad JPA en la API

Riesgo / CVSS v3.1	Severidad: Media · Base Score: 5.3 CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:N
Hallazgo	La respuesta del registro expone directamente la entidad JPA (incluyendo id, periodo y marcadaRevision), filtrando el modelo interno. @Data genera setters públicos y equals/hashCode problemáticos en entidades JPA.
Activos afectados	AporteController.registrar() devuelve la entidad Aporte; entidades anotadas con @Data.
Escenario de riesgo	El consumidor recibe más campos de los necesarios y el modelo de persistencia queda acoplado al contrato público; futuros cambios de BD rompen la API.
Impacto	Filtración del modelo interno y acoplamiento; riesgo de sobre-exposición en respuestas.
Probabilidad	Media.
Clasificación	OWASP: A08:2021 – Fallos de integridad de software y datos · CWE: CWE-915 (Modificación indebida de atributos; «mass assignment»); relacionado con CWE-213 · ATT&CK: No aplica
Validación	Verdadero positivo. Evidente por lectura de código.
Descripción y causa raíz	No se separa el modelo de dominio/persistencia del modelo de transporte en la respuesta.
Superficie de ataque	Cuerpo de respuesta del POST.
Acciones correctivas	Inmediata: Devolver un DTO de respuesta explícito y no exponer entidades JPA. Estructural: Convención de mapeo entidad↔DTO y revisión de equals/hashCode en entidades.

Fragmento relevante

```
@PostMapping
public Aporte registrar(@RequestBody AporteRequest req) { // expone la entidad
    return service.registrar(req);
}
```

A-12 · Violación de límites arquitectónicos (Clean Architecture / CQRS)

Riesgo / CVSS v3.1	Severidad: Media · Base Score: 4.0 N/A (deuda técnica · defensa en profundidad)
Hallazgo	La consulta accede directamente a la base de datos desde el controlador, saltándose servicio y repositorio, mientras el comando sí pasa por el servicio: no hay una separación comando/consulta coherente ni CQRS. Las reglas de negocio viven en el servicio y no en el dominio, y el servicio depende de repositorios concretos en lugar de puertos.
Activos afectados	AporteController (usa JdbcTemplate y un RowMapper propio) y capa domain (entidades JPA anémicas).
Escenario de riesgo	No es explotable por sí mismo, pero erosiona los límites del CIS (Clean Architecture, SOLID, CQRS) y facilita que defectos como A-01 aparezcan.
Impacto	Deuda técnica, acoplamiento y mayor superficie para errores; dificulta pruebas y mantenimiento.
Probabilidad	N/A (calidad/diseño).
Clasificación	OWASP: A04:2021 – Diseño inseguro · CWE: CWE-710 (Incumplimiento de estándares de codificación); relacionado con CWE-1061 · ATT&CK: No aplica
Validación	Verdadero positivo (observación de diseño).
Descripción y causa raíz	La responsabilidad de acceso a datos está dispersa y el dominio es anémico.
Superficie de ataque	Estructura de capas del módulo.
Acciones correctivas	Inmediata: Mover la consulta al repositorio/servicio; que el controlador no hable con la base de datos. Estructural: Definir puertos de entrada/salida, separar comando y consulta, y concentrar las reglas en el dominio.

A-13 · Trazabilidad y auditoría débiles del evento

Riesgo / CVSS v3.1	Severidad: Media · Base Score: 4.3 CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:L/A:N
Hallazgo	El evento se guarda antes que el aporte y sin el id de este, por lo que no se pueden correlacionar; el campo tipo es un String libre; y fechaEvento usa LocalDateTime.now() sin zona horaria.
Activos afectados	EventoAporte.java y AporteService.java · línea 62.
Escenario de riesgo	Ante una investigación o auditoría SFC, no es posible enlazar de forma fiable un evento con su aporte ni fijar el instante exacto.
Impacto	Auditoría y trazabilidad insuficientes para un entorno regulado.
Probabilidad	N/A (observabilidad).
Clasificación	OWASP: A09:2021 – Fallos de registro y monitoreo · CWE: CWE-778 (Registro insuficiente); relacionado con CWE-779 · ATT&CK: No aplica
Validación	Verdadero positivo. Orden de persistencia evidente por lectura de código.
Descripción y causa raíz	El registro de eventos existe (buena intención) pero carece de los enlaces y metadatos necesarios para ser útil.
Superficie de ataque	Modelo de evento y su persistencia.
Acciones correctivas	Inmediata: Persistir el aporte primero y referenciar su id en el evento; usar un enum para tipo y un instante con zona (Instant / OffsetDateTime). Estructural: Definir un modelo de auditoría con correlación de extremo a extremo y retención acorde a la normativa.

4. Conclusiones

Fortalezas

- El módulo compila y los tests «felices» pasan; existe una estructura por capas (controlador, servicio, repositorios) y un DTO de entrada (AporteRequest).
- Hay una intención de auditoría mediante el registro de eventos (EventoAporte) y un umbral de marcado para revisión posterior, acorde con el espíritu del requisito.

Debilidades

- Seguridad: inyección SQL (A-01) y ausencia total de autenticación/autorización (A-02).
- Integridad del dinero: double para montos (A-04), tope comparado con «==» (A-03), falta de transaccionalidad y concurrencia (A-05), registro no idempotente (A-06) y saldo no mensual (A-07).
- Calidad y cumplimiento: manejo de errores deficiente (A-09), logging de datos sensibles (A-10), exposición de la entidad JPA (A-11), límites arquitectónicos rotos (A-12) y trazabilidad débil (A-13).

Veredicto del Merge Request

No aprobar en su estado actual. Los hallazgos A-01 a A-08 (los dos críticos y los seis altos) son bloqueantes: comprometen la seguridad y la integridad del dinero y deben corregirse antes de fusionar. Los medios A-09 a A-13 deben planificarse en el mismo ciclo.

Qué falta para ser apto para producción en un entorno SFC

- Seguridad: autenticación y autorización, gestión de secretos y validación estricta del lado del servidor.
- Integridad financiera: BigDecimal con escala fija, restricciones en base de datos (NOT NULL, CHECK monto > 0, unicidad por (afiliado, mes)) e idempotencia a escala.
- Consistencia: transacciones y control de concurrencia (bloqueo optimista/pesimista) en la actualización del saldo.
- Observabilidad y trazabilidad: correlación evento↔aporte, marcas de tiempo con zona y logs sin datos sensibles.
- Entrega: migraciones versionadas (Flyway) en lugar de ddl-auto=create-drop, y pruebas que cubran tope, concurrencia e idempotencia (hoy ausentes).